

✓ University of Utah CS 6340/5340 NLP Fall 2024 Assignment 4

First, make a copy of this notebook: File > Save a copy in Drive.

Connect to a GPU by clicking on the arrow next to "Connect" in the upper right corner, then click "Change runtime type" and select "T4 GPU".

Turn off the code completion by going to Tools > Settings > Editor > Automatically trigger code completions > Uncheck the box next to it. **Using the completed code is the same as copying it which is the academic misconduct in this class.**

Run the cells with your solutions such that the output is visible. When you are ready to submit your solution to Gradescope, do the following:
File > Print > Destination: Save as PDF > Save. Upload the pdf to Gradescope.

✓ Quantization (40 points)

You will implement and apply quantization equations provided in the class to a floating-point matrix to convert it to an 8-bit signed integer matrix and then reconstruct the original weights. You will also analyze the memory usage and the quantization error. This will help you understand how quantization works and its impact on memory and precision.

Run the following cell to construct a 5 x 5 matrix with random numbers from a normal distribution with mean 0 and variance 1, stored in 32-bit float precision.

```
import torch

original_matrix = torch.randn(5, 5, dtype=torch.float32)
print("A matrix (32-bit float):\n", original_matrix)

↗ A matrix (32-bit float):
  tensor([[ 0.0504,  0.1824,  0.9829,  0.9935, -1.0701],
         [-0.3815,  0.6163, -0.7323, -0.7072,  0.5910],
         [-0.6762, -0.4973, -1.9598, -0.9373,  0.7608],
         [-1.0795,  2.0767,  0.9751, -0.2037, -1.7776],
         [ 0.4013, -1.1736,  1.1849,  0.4607, -0.3446]])
```

Use the formulas provided in the class to compute the scale factor (s) and zero-point (z). Print these two values.

Tip: The range of 8-bit signed integers is -128 to 127.

```
q_min = -128
q_max = 127
x_min = original_matrix.min().item()
x_max = original_matrix.max().item()

s = (q_max - q_min) / (x_max - x_min)
z = q_min - round(s * x_min)

print(f"Scale factor (s): {s}")
print(f"Zero-point (z): {z}")

↗ Scale factor (s): 63.1732656711467
  Zero-point (z): -4
```

Quantize the original matrix to 8-bit integers using the calculated scale factor and zero-point. Name the quantized matrix `quantized_matrix`.

Tips:

1. You are allowed to use `float32` temporarily.
2. You can use `.to(torch.int8)` after rounding.

```
quantized_matrix = torch.round((original_matrix * s) + z).to(torch.int8)
print("Quantized matrix (8-bit integers):\n", quantized_matrix)
```

```
↗ Quantized matrix (8-bit integers):
  tensor([[ -1,   8,  58,  59, -72],
         [-28,  35, -50, -49,  33],
         [-47, -35, -128, -63,  44],
```

```
[ -72, 127, 58, -17, -116],
[ 21, -78, 71, 25, -26]], dtype=torch.int8)
```

Implement and use the formula to dequantize the matrix back to 32-bit floats. Name the reconstructed matrix `reconstructed_matrix`.

Start coding or [generate](#) with AI.

```
reconstructed_matrix = ((quantized_matrix.to(torch.float32) - z)/s).to(torch.float32)
print("Reconstructed matrix (32-bit floats):\n", reconstructed_matrix)
```

```
↗ Reconstructed matrix (32-bit floats):
tensor([[ 0.0475,  0.1900,  0.9814,  0.9973, -1.0764],
        [-0.3799,  0.6173, -0.7282, -0.7123,  0.5857],
        [-0.6807, -0.4907, -1.9629, -0.9339,  0.7598],
        [-1.0764,  2.0737,  0.9814, -0.2058, -1.7729],
        [ 0.3957, -1.1714,  1.1872,  0.4591, -0.3482]])
```

Run the following cell to get the the total memory consumption in bytes for each matrix.

- `element_size()` gives the size of each element in bytes (4 bytes for float32, 1 byte for int8).
- `numel()` returns the total number of elements in the matrix. By multiplying these two, we get the total memory consumption in bytes for each matrix.

```
original_memory = original_matrix.element_size() * original_matrix.numel()
quantized_memory = quantized_matrix.element_size() * quantized_matrix.numel()
```

```
print(f"Memory usage (original matrix): {original_memory} bytes")
print(f"Memory usage (quantized matrix): {quantized_memory} bytes")
```

```
↗ Memory usage (original matrix): 100 bytes
Memory usage (quantized matrix): 25 bytes
```

Run the following cell to get the reconstruction/quantization error.

- Mean Squared Error (MSE) is a standard metric to measure the difference between two matrices element-wise.

```
mse = torch.mean((original_matrix - reconstructed_matrix) ** 2)
print(f"Quantization Error: {mse.item()}")
```

```
↗ Quantization Error: 1.700960456219036e-05
```

In the following text field provide answers to these questions:

1. In your view, is the quantization error low? Why or why not?
2. Is the reduction in memory usage significant?
3. How might these changes (quantization error and memory reduction) affect the performance and efficiency of the model during inference?

Be concise and precise.

✓ Answer

1. I think the quantization error is very low. This means that the difference between the original matrix and the reconstructed matrix is very less.
2. I think it is a very significant memory reduction, it went from 100 bytes to 25 bytes which is a 75% reduction. This would help in the devices which have low memory.
3. As we can see this is a very good optimization technique, the quantization error and memory usage both are low, this would increase the performance and the efficiency of the model.

✓ KV Cache (40 points)

Key-Value (KV) caching is used for speeding up inference with decoder-only transformers that predict one token at the time.

Take some time to read through the code in the next cell and run it. Try to figure out what the code is doing. Once you've done that, explain the differences in the reported times and why the tensors match even though the times differ. **Be concise and precise.**

```
import torch
import time

seq_len = 2000
d_model = 768
d_internal = 64

W_k = torch.randn(d_model, d_internal, dtype=torch.float32)
W_v = torch.randn(d_model, d_internal, dtype=torch.float32)
def compute_kv(embedding):
    K = embedding @ W_k
    V = embedding @ W_v
    return K, V

keys_list = []
values_list = []
def store_kv(new_token):
    new_K, new_V = compute_kv(new_token)
    keys_list.append(new_K)
    values_list.append(new_V)

input_embeddings = torch.randn(seq_len, d_model, dtype=torch.float32)

start = time.time()
for i in range(seq_len):
    K1, V1 = compute_kv(input_embeddings[i + 1])
end = time.time()
print(f"\nTime to compute K,V with Method 1: {end - start:.6f} seconds")

start = time.time()
for i in range(seq_len):
    store_kv(input_embeddings[i])
    K2 = torch.stack(keys_list)
    V2 = torch.stack(values_list)
end = time.time()
print(f"Time to compute K,V with Method 2: {end - start:.6f} seconds")

keys_match = torch.allclose(K1, K2, atol=1e-4, rtol=1e-3)
values_match = torch.allclose(V1, V2, atol=1e-4, rtol=1e-3)
print(f"\nKey tensors match: {keys_match}")
print(f"Value tensors match: {values_match}")
```



```
Time to compute K,V with Method 1: 6.817197 seconds
Time to compute K,V with Method 2: 1.989629 seconds
```

```
Key tensors match: True
Value tensors match: True
```

In the following text field provide your explanation.

✓ Answer

Time: We can see that the first method takes a lot of time meanwhile the second method takes less time. First method generates key value pair for each token, everytime until the for loop ends.

Meanwhile the second method generates the key value pair for the current token and stores it. In this it doesn't need to recompute for tokens it has already processed, hence this method is faster.

Tensor: Even though both methods have difference in time, the result is same in both cases as key value pair are generated for each and every token. Hence the tensor would match but time would differ.

✓ Finetuning an LLM on SQuAD using Huggingface's libraries (20 points + Bonus)

[Huggingface](#) provides an ecosystem of pretrained language models, datasets, tokenizers, and other utilities. It simplifies the process of working with state-of-the-art NLP models and allows researchers and developers to finetune LLMs efficiently. Huggingface's transformers, datasets, and accelerate libraries are widely used across academia and industry for NLP research and deployment. **Experience with Huggingface is important if you plan to work in NLP in the future.**

[The Stanford Question Answering Dataset \(SQuAD\)](#) is a benchmark dataset for reading comprehension task. The goal is to predict the answer to a question from a given context passage. This passage is provided, not retrieved. SQuAD has been popular for several reasons:

- It encourages the development of models that understand context at a somewhat deeper level. This has been challenging for the state-of-the-art models when the time when this dataset was introduced.
- It provides clear evaluation metrics, token-level F1 and exact match (EM) scores, which are easy to calculate.
- It has consequently driven significant advancements.

In the next text cell, write about possible modeling choices for finetuning an LLM on SQuAD. Consider possible output layers, transformer types, model size, relevance of the model is instruction finetune or aligned, maximum input sequence length, etc. Comment on pros and cons. After you consider various possibilities, explain which approach you would choose and why.

✓ Answer

There are many choices to choose from.

1. BERT: Bi-directional transformer would be a great choice to understand context within sentences.
2. RoBERTa: Improved BERT as it works on a larger dataset but would require more training steps.
3. Transformer(T4): Transformer model would work for question answering .
4. GPT: Decoder model.

My choice:

BERT and RoBERTa are the transformer type architecture and would be suitable for this task. They are trained on a larger dataset and would perform better.

Pros: A very good balance between accuracy and computational power. Light-weight model works well for extraction task.

Cons: Context is limited to the current passage.

We would have 2 output layers, one for the start and one for the end position of the answer.

Pros: Effective for getting answer in the span of text.

Cons: Wouldn't perform better in the case of combining information across multiple spans.

BERT and RoBERTa would be a decent in terms of model size. A larger option would be BERT large and RoBERT large but they would require more training data and would take more time. So choosing normal versions of BERT and RoBERTa are tradeoff for accuracy and complex tasks and time taken.

Pros: Takes less time to train.

Cons: Might be inaccurate.

Size can be 512. Smaller size like 64 or 128 won't be sufficient. A larger size like 1024 would be expensive.

In [this notebook](#) you can find the code for finetuning an LM to *extract* a span in a given text to answer a question. Please go through this code carefully. You can either prompt ChatGPT to explain portions of that code or ask us in Piazza. In the next part, you are going to approach reading comprehension in SQuAD by *generating* the answer, not extracting it.

✓ Finetuning the Qwen2.5 language model on SQuAD to *generate* answers

Start coding or [generate](#) with AI.

✓ Analysis of RAM usage (20 points)

In the following code cell you are going to write the code to load the tokenizer and weights of a Qwen2.5 model. In the figure below, you can find sizes in which this model comes. The name of the Qwen2.5 family of models in Huggingface is [Qwen/Qwen2.5-XB-Instruct](#) where X should be replaced by a concrete number of parameters. Use the Auto class which does **not** make a new classification head for the task.

Start from the smallest model and load the weights, then restart the session (Runtime > Restart session or Cmd/Ctrl+M), and try to load the next larger model. At some point you will get:

Your session crashed after using all available RAM.

In the next text cell report for which model size do you get this error and explain why do you get this error.

Models	# Params (B)	# Non-Emb Params (B)	# Layers	# Head (Q / KV)	# Tie Embedding	Context Length	Generation Length	License
Qwen2.5-0.5B	0.49	0.36	24	14/2	Yes	32K	8K	Apache 2.0
Qwen2.5-1.5B	1.5	1.3	28	12/2	Yes	32K	8K	Apache 2.0
Qwen2.5-3B	3.1	2.8	36	16/2	Yes	32K	8K	Qwen Research
Qwen2.5-7B	7.6	6.5	28	28/4	No	128K	8K	Apache 2.0
Qwen2.5-14B	14.7	13.1	48	40/8	No	128K	8K	Apache 2.0
Qwen2.5-32B	32.5	31.0	64	40/8	No	128K	8K	Apache 2.0
Qwen2.5-72B	72.7	70.0	80	64/8	No	128K	8K	Qwen
Qwen2.5-Coder-1.5B	1.5	1.3	28	12/2	Yes	128K	2K	Apache 2.0
Qwen2.5-Coder-7B	7.6	6.5	28	28/4	No	128K	2K	Apache 2.0
Qwen2.5-Math-1.5B	1.5	1.3	28	12/2	Yes	4K	2K	Apache 2.0
Qwen2.5-Math-7B	7.6	6.5	28	28/4	No	4K	2K	Apache 2.0
Qwen2.5-Math-72B	72.7	70.0	80	64/8	No	4K	2K	Qwen

```
import torch
from transformers import AutoTokenizer, AutoModel
device = "cuda"
print(f"Running on: {device}")
model_names = [
    # "Qwen/Qwen2.5-0.5B",
    # "Qwen/Qwen2.5-1.5B",
    "Qwen/Qwen2.5-3B",
    "Qwen/Qwen2.5-7B",
    "Qwen/Qwen2.5-14B",
    "Qwen/Qwen2.5-32B",
    "Qwen/Qwen2.5-72B"
]
for model_name in model_names:
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModel.from_pretrained(model_name, device_map="cuda")
    print(f"model:{model_name}")
```

Running on: cuda
 /usr/local/lib/python3.10/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
 The secret `HF\_TOKEN` does not exist in your Colab secrets.
 To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set
 You will be able to reuse this secret in all of your notebooks.
 Please note that authentication is recommended but still optional to access public models or datasets.
 warnings.warn()

tokenizer_config.json: 100%	7.23k/7.23k [00:00<00:00, 227kB/s]
vocab.json: 100%	2.78M/2.78M [00:00<00:00, 6.99MB/s]
merges.txt: 100%	1.67M/1.67M [00:00<00:00, 5.26MB/s]
tokenizer.json: 100%	7.03M/7.03M [00:00<00:00, 26.4MB/s]
config.json: 100%	683/683 [00:00<00:00, 24.9kB/s]
model.safetensors.index.json: 100%	35.6k/35.6k [00:00<00:00, 1.27MB/s]
Downloading shards: 100%	2/2 [02:33<00:00, 72.80s/it]
model-00001-of-00002.safetensors: 100%	3.97G/3.97G [01:40<00:00, 41.5MB/s]
model-00002-of-00002.safetensors: 100%	2.20G/2.20G [00:53<00:00, 39.6MB/s]
Loading checkpoint shards: 100%	2/2 [00:32<00:00, 15.38s/it]
model: Qwen/Qwen2.5-3B	
tokenizer_config.json: 100%	7.23k/7.23k [00:00<00:00, 380kB/s]
vocab.json: 100%	2.78M/2.78M [00:00<00:00, 28.0MB/s]
merges.txt: 100%	1.67M/1.67M [00:00<00:00, 4.98MB/s]
tokenizer.json: 100%	7.03M/7.03M [00:00<00:00, 16.0MB/s]
config.json: 100%	686/686 [00:00<00:00, 48.0kB/s]
model.safetensors.index.json: 100%	27.8k/27.8k [00:00<00:00, 1.56MB/s]
Downloading shards: 25%	1/4 [01:42<05:08, 102.70s/it]
model-00001-of-00004.safetensors: 100%	3.95G/3.95G [01:42<00:00, 38.2MB/s]
model-00002-of-00004.safetensors: 17%	671M/3.86G [00:17<01:16, 41.5MB/s]

```
import torch
from transformers import AutoTokenizer, AutoModel
device = "cpu"
print(f"Running on: {device}")
model_names = [
    "Qwen/Qwen2.5-0.5B",
    "Qwen/Qwen2.5-1.5B",
    "Qwen/Qwen2.5-3B",
    "Qwen/Qwen2.5-7B",
    "Qwen/Qwen2.5-14B",
    "Qwen/Qwen2.5-32B",
    "Qwen/Qwen2.5-72B"
]
for model_name in model_names:
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModel.from_pretrained(model_name, device_map="cpu")
    print(f"model:{model_name}")
```



```

Running on: cpu
model: Qwen/Qwen2.5-0.5B
tokenizer_config.json: 100% 7.23k/7.23k [00:00<00:00, 141kB/s]
vocab.json: 100% 2.78M/2.78M [00:01<00:00, 2.66MB/s]
merges.txt: 100% 1.67M/1.67M [00:00<00:00, 1.90MB/s]
tokenizer.json: 100% 7.03M/7.03M [00:00<00:00, 33.7MB/s]
config.json: 100% 684/684 [00:00<00:00, 20.9kB/s]
model.safetensors: 100% 3.09G/3.09G [01:34<00:00, 33.2MB/s]
model: Qwen/Qwen2.5-1.5B
tokenizer_config.json: 100% 7.23k/7.23k [00:00<00:00, 288kB/s]
vocab.json: 100% 2.78M/2.78M [00:00<00:00, 16.0MB/s]
merges.txt: 100% 1.67M/1.67M [00:00<00:00, 1.92MB/s]
tokenizer.json: 100% 7.03M/7.03M [00:00<00:00, 18.0MB/s]
config.json: 100% 683/683 [00:00<00:00, 26.5kB/s]
model.safetensors.index.json: 100% 35.6k/35.6k [00:00<00:00, 2.27MB/s]
Downloading shards: 100% 2/2 [03:41<00:00, 106.64s/it]
model-00001-of-00002.safetensors: 100% 3.97G/3.97G [02:14<00:00, 36.3MB/s]
model-00002-of-00002.safetensors: 100% 2.20G/2.20G [01:26<00:00, 7.99MB/s]
Loading checkpoint shards: 0% 0/2 [00:00<?, ?it/s]

```

✓ Answer

Running both on CPU and GPU, I run out of memory at 3B.

A parameter would require 4 bytes of memory. 3B means 3 Billion instructions. This would require atleast 12GB of RAM, that is why it crashes while loading 3B, for both CPU and GPU as we have a limit at 12GB.

I tried only running 3B and it still did not load.

✓ Bonus (max. 15 points)

In this part, you can earn additional points by finetuning and evaluating a Qwen model on SQuAD v1 to generate *not extract* answers in the following two ways:

1. Finetuning with full precision. You will need to select an appropriate model size.
2. Finetuning using QLoRA.

Please note that this section requires more effort than previous parts. Given the circumstances, I've decided to make the fourth assignment easier and turn this section into a bonus. Awarding too many points for the bonus part could inflate class grades, so I will award 15 points at max for it. If you're excited about NLP, I strongly encourage you to complete this part.

As always, you are **not** allowed to directly prompt ChatGPT, Gemini, Claude, Copilot, or other LLMs to write the solution for you. However, you are permitted to seek help by analyzing related code available on the web or Huggingface resources. This mirrors how you would use Huggingface in real-world applications. However, **you must report all resources used at the end of this notebook!** Failing to do so is considered academic misconduct. You may also seek help from LLMs to explain errors in your code. **However, you must report any LLM usage for this purpose and include your prompts!** Failing to report this is also considered academic misconduct.



We will conduct **one-on-one interviews** with anyone who submits their solutions to this part to probe their understanding of the submitted code and related concepts. There will be no penalty for not demonstrating sufficient understanding, but we may withhold points if there is a significant lack of understanding of the code and concepts. This addition to evaluation is intended to avoid awarding submission of an LLM's solution, which is challenging to detect. 

Other requirements:

- You must load the SQuAD dataset using the [datasets](#) library.
- You must use [AutoTokenizer](#) to load the tokenizer.
- Use [map\(\)](#) to tokenize the entire dataset efficiently.

- You must report the token-F1 and exact match for the validation split of the data.
- Use Huggingface's [Trainer](#) to finetune your model.
- You can explore using a sample of training data for finetuning.

These resources might be helpful:

- [Check "QLoRa, an even more efficient method" here](#)
- [The HF notebook that goes through the recent bitsandbytes integration](#)
- [HF Transformers Notebooks](#)

Tip:

- I find wandb distracting and disable it with `import os; os.environ["WANDB_MODE"] = "disabled"`.

Please provide your implementation and run the cells such that we can see the token-f1 and exact match on the validation set.

Start coding or [generate](#) with AI.

✓ Report of resources and prompts used